```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


# Construct expression tree from prefix expression
def construct_tree(prefix):
    stack = []

    for symbol in reversed(prefix):
        if symbol.isalnum():
            stack.append(Node(symbol))
        else:
            node = Node(symbol)

            node.left = stack.pop()
            node.right = stack.pop()

            stack.append(node)

    return stack.pop()


# Non-recursive post-order traversal
def postorder(root):
    if root is None:
        return

    stack1 = [root]
    stack2 = []

    while stack1:
        node = stack1.pop()
        stack2.append(node)

        if node.left:
            stack1.append(node.left)

        if node.right:
            stack1.append(node.right)

    while stack2:
        print(stack2.pop().data, end=" ")


# Delete the entire tree
def delete_tree(root):
```

```
    if root is None:
        return None

    stack = [root]

    while stack:
        node = stack.pop()

        if node.left:
            stack.append(node.left)

        if node.right:
            stack.append(node.right)

        node.left = None
        node.right = None

    return None


# Main program
prefix = "+-a*bc/def"

root = construct_tree(prefix)

print("Post-order traversal:")
postorder(root)

root = delete_tree(root)

print("\n\nTree deleted successfully.")
print("Root after deletion:", root)
```